

LOCAL LLM TEXT GENERATION USING STREAMLIT

Question 1 – Implementation

1. Question

Develop a Streamlit application for text generation using a locally running Large Language Model.

2. Aim

To develop a user-friendly Streamlit web application that accepts a text prompt and generates a response using a locally running Large Language Model (LLM) through Ollama and Python.

3. Technologies Used

- Python 3.12.10
- Streamlit – for developing the interactive web interface
- Ollama 0.33.2 – for running the LLM locally
- Llama 3.2 – local Large Language Model
- VS Code – development environment

4. Project Folder Structure

Local_LLM_Text_Generation/

├── venv/ – Python virtual environment

└── app.py – Streamlit application

5. Implementation Steps

1. Create and open the Local_LLM_Text_Generation folder in VS Code.
2. Create a Python virtual environment using: `python -m venv venv`
3. Activate the environment using: `.\venv\Scripts\Activate.ps1`
4. Install the required packages using: `pip install streamlit ollama`
5. Install Ollama and verify it using: `ollama --version`
6. Download the local Llama 3.2 model using: `ollama pull llama3.2`
7. Test the model using: `ollama run llama3.2`
8. Create app.py and connect the Streamlit interface to Ollama using the Python Ollama library.
9. Run the application using: `streamlit run app.py`
10. Open the local Streamlit URL, enter a prompt, and click Generate Text to obtain the response.

6. Core Python Implementation

```
import streamlit as st
import ollama

st.set_page_config(
    page_title="Local LLM Text Generator",
    page_icon="🤖",
    layout="centered"
)

st.title("🤖 Local LLM Text Generator")
```

```

st.write(
    "Generate text using a locally running Large Language Model "
    "through Ollama."
)

model = st.selectbox(
    "Select Local LLM",
    ["llama3.2"]
)

prompt = st.text_area(
    "Enter your prompt",
    placeholder="Example: Explain Artificial Intelligence in simple words.",
    height=150
)

if st.button("Generate Text", type="primary", width="stretch"):
    if not prompt.strip():
        st.warning("Please enter a prompt.")
    else:
        response = ollama.chat(
            model=model,
            messages=[
                {"role": "user", "content": prompt}
            ]
        )

        result = response["message"]["content"]

        st.subheader("Generated Text")
        st.write(result)

```

7. Working Principle

The user enters a prompt in the Streamlit interface. The Python application sends the prompt to Ollama, which passes it to the locally running Llama 3.2 model. The generated response is returned to Python and displayed on the Streamlit page.

8. System Workflow

11. User enters a text prompt
12. Streamlit captures the prompt
13. Python sends the prompt to Ollama
14. Ollama invokes the local Llama 3.2 model
15. The model generates the response
16. The response is returned to the Python application
17. Streamlit displays the generated text

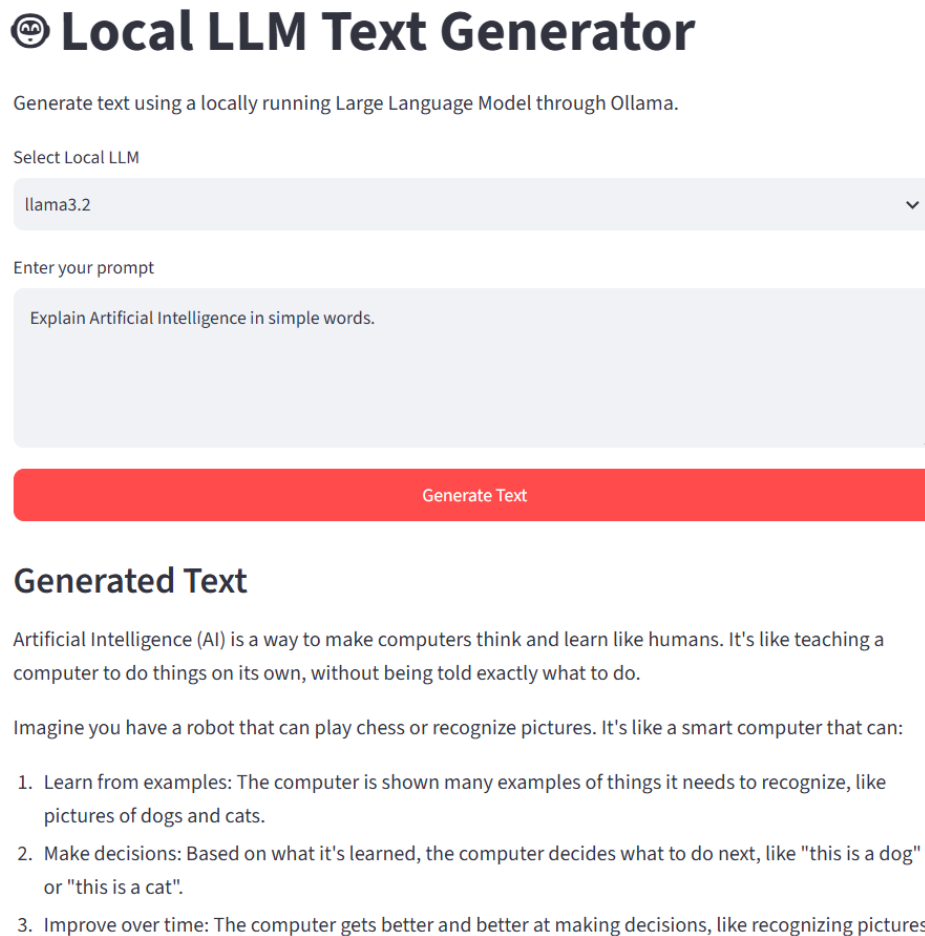
9. Input

Prompt: Explain Artificial Intelligence in simple words.

10. Output

The application generates an explanatory response from the local Llama 3.2 model. The response is displayed under the heading “Generated Text” in the Streamlit interface.

11. Application Output Screenshot



The screenshot shows a web interface titled "Local LLM Text Generator". Below the title is a subtitle: "Generate text using a locally running Large Language Model through Ollama." There is a dropdown menu labeled "Select Local LLM" with "llama3.2" selected. Below this is a text input area labeled "Enter your prompt" containing the text "Explain Artificial Intelligence in simple words." A red button labeled "Generate Text" is positioned below the input area. Underneath the button, the heading "Generated Text" is displayed, followed by the generated response: "Artificial Intelligence (AI) is a way to make computers think and learn like humans. It's like teaching a computer to do things on its own, without being told exactly what to do. Imagine you have a robot that can play chess or recognize pictures. It's like a smart computer that can:" followed by a numbered list of three points: 1. Learn from examples: The computer is shown many examples of things it needs to recognize, like pictures of dogs and cats. 2. Make decisions: Based on what it's learned, the computer decides what to do next, like "this is a dog" or "this is a cat". 3. Improve over time: The computer gets better and better at making decisions, like recognizing pictures.

Figure 1: Streamlit interface showing local LLM text generation

12. Result

Thus, a Streamlit application for text generation was successfully developed and executed using a locally running Large Language Model through Ollama and Python. The application accepts user prompts and displays the generated response through an interactive web interface.

13. Conclusion

The implementation demonstrates how a local LLM can be integrated with a Python and Streamlit application without relying on a cloud-based language model API. Ollama provides the local model runtime, while Streamlit provides a simple and interactive interface for text generation.